

An interactive type checker for dependent types with general recursion (System Description)

Valentin Pasquale

Université Paris-Saclay, CEA, List, F-91120
Palaiseau, France
valentin.pasquale@cea.fr

Álvaro García-Pérez

Université Paris-Saclay, CEA, List, F-91120
Palaiseau, France
alvaro.garciaperez@cea.fr

Abstract

In this system description, we present an interactive type-checker for Hutchins’ Pure Subtype Systems (PSS). By blurring the distinction between types and terms, PSS features dependent types and general recursion, but the resulting theory is highly impredicative and not strongly normalising, which poses a number of known challenges for practical implementations of type-checking. We propose an interactive type-checker where the programmer drives the type-checking process by performing different actions on the program term. Our interactive type-checker is based on Emacs and is written in Emacs Lisp. We provide a motivating example involving the type-checking of a safe division function, and we show how our type-checker certifies this function is safe by implementing a rudimentary abstract interpretation on top of PSS type-checking.

ACM Reference Format:

Valentin Pasquale and Álvaro García-Pérez. 2025. An interactive type checker for dependent types with general recursion (System Description). In *Proceedings of the 27th International Symposium on Principles and Practice of Declarative Programming (PPDP ’25)*, September 10–11, 2025, Rende, Italy. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3756907.3756925>

1 Introduction

In this system description, we present an interactive type-checker for Hutchins’ Pure Subtype Systems (PSS for short), which are based on a novel and intriguing typing discipline that blurs the distinction between types and terms [12].¹ PSS (hereafter in the singular) features higher-order subtyping, dependent types, and general recursion. These features combined allow for great expressivity and enable a number of advanced language features for genericity and extensibility. However, PSS is highly impredicative and not strongly normalising, which poses a number of known metatheoretical challenges, the most relevant for our work here being that type-checking in PSS is undecidable in general.²

Our type-checker circumvents this undecidability by letting the programmer drive the type-checking process. Our system allows one to type-check PSS terms, where some parts of the algorithm

are automatic, and other parts rely on the interactions with the programmer, who likely knows how to type-check their program.

Consider the following program:

$$\begin{aligned} \text{fact}(0) &= 1 \\ \text{fact}(n+1) &= (n+1) * \text{fact}(n) \\ \text{safeDiv}(n, m) &= n / \text{fact}(m) \end{aligned}$$

With PSS, our type-checker could certify that *safeDiv* never incurs a division-by-zero error, since the factorial function always returns a result greater than one. Certifying this static guarantee is not direct in conventional type systems (like Haskell or ML). We provide suitable encodings for the naturals and show how to implement a rudimentary abstract interpretation in PSS in a straightforward fashion. Thanks to PSS impredicativity, we can apply the successor function to the type of naturals, thus obtaining a domain of interpretation that represents all the naturals produced by the successor function, which are exactly those that are greater than zero. Then we show that the value returned by the factorial function is a subtype of the said domain of interpretation.

Our focus on recursive implementations of factorial, which is itself primitive recursive, showcases the ability of our approach to be generalised to general recursive functions, and functions that may not terminate. Consider the program below:

$$\begin{aligned} \text{Syracuse}(0) &= 1 \\ \text{Syracuse}(1) &= 1 \\ \text{Syracuse}(n) &= \text{Syracuse}(n/2) && (\text{if } n \text{ is even}) \\ \text{Syracuse}(n) &= \text{Syracuse}((3 * n) + 1) && (\text{otherwise}) \\ \text{safeDiv2}(n, m) &= n / \text{Syracuse}(m) \end{aligned}$$

It is a variation of the Syracuse function, which is traditionally not defined for 0. It is not known whether *Syracuse* terminates on every natural number or not. However, if the function terminates, then it always returns 1. Our type-checker can also certify that *safeDiv2* above is safe.

We focus our discussion mainly on the recursive implementation of *fact*—rather than on the implementation of *Syracuse*—because the former involves applying arithmetic operations on its recursive call, which is not the case for the type-checking derivation of the latter.³

In a superficial way, our type-checker resembles an interactive theorem prover. However, due to the lack of strong normalisation, whether PSS corresponds to some logic by the Curry-Howard isomorphism is still an open problem, and as a consequence our type-checker is not suited for doing mechanised mathematics right away.

¹Our implementation is open source and available on GitHub at the following address: <https://github.com/valentin-ppp/interactive-dependenttypes-typechecker>.

²In a related ongoing work, the authors study PSS meta-theory, notably its type safety and the conditions on which type-checking is decidable.



This work is licensed under a Creative Commons Attribution 4.0 International License. *PPDP ’25, Rende, Italy*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2085-7/25/09

<https://doi.org/10.1145/3756907.3756925>

³We cover the type-checking of *Syracuse* in Section 4.5, but this is not central to the bulk of our paper

The remainder of the paper is organised as follows. Section 2 recalls PSS (with minor modifications that we comment on in detail) and the encodings of different objects including the naturals, which we dub "integers" as conventionally in programming. Section 3 comments on our design and implementation choices, and describes how our interactive type-checker is used. Section 4 showcases the type-checking of different programs, including those in the motivating examples above. Section 5 concludes by enumerating related work and by discussing avenues for future work.

2 Preliminaries

In this section, we present the theory we use, which is a slight variation of the original PSS theory. We then present how we encode objects in PSS, and finally, we detail the changes we made to the original PSS theory.

2.1 The PSS theory

PSS blurs the distinction between types and terms and has numerous interesting features explained at length in Hutchins' PhD thesis [12], which we do not explore in this paper. Type-checking is known to be challenging. In this paper, we focus only on practical approaches to type-checking PSS programs. PSS is an extension of the pure lambda calculus with a constant symbol `Top` for the most general supertype, and with subtype annotations in the formal parameters of abstractions. Assume a countably infinite set of variables V ranged over by $x, y, \dots$; the syntax of terms and contexts of PSS, extended with a fixed point combinator and a variant type (which we describe in a moment), is given by the following grammar:

$$\begin{aligned} t, u, v, \dots \in \Lambda &::= x \mid \text{Top} \mid (\lambda x \leq t.u) \mid (uv) \mid (Y u) \mid \text{Or}(u, v) \\ \Gamma &::= \varepsilon \mid \Gamma, x \leq t \end{aligned}$$

The type-checking process of PSS ensures that for every application (uv) of a given term, v is a subtype of the type annotation of a function of which u is a subtype.

In our theory, the program we wish to type-check is the following:

$$\begin{aligned} \text{fact} &= Y \lambda s \leq (\lambda x \leq \text{int.int}). \lambda n \leq \text{int}. \\ &\quad \text{match } n \text{ one } (\lambda p \leq \text{int.mult } n (s p)) \\ \text{safediv} &= \lambda n \leq \text{int}. \lambda m \leq \text{int}. \text{div } n (\text{fact } m) \\ \text{div} &= \lambda n \leq \text{int}. \lambda m \leq (\text{succ int}). \dots \end{aligned}$$

where *one* and *int* are respectively the term for the integer 1 and for the integer type, *mult* and *succ* stand for the arithmetic operations of the same names, and *match* implements the pattern matching on an integer being either 0 or a successor. The expressions of these terms are defined in Section 2.2.

In this theory, we consider a static condition of *well-formedness* that prescribes that every application in a term is well-typed, and it is an open problem to show it ensures type-safety.⁴ Verifying the well-formedness of a term is akin to type-checking a term in other theories. In the rest of the paper, we refer to the process of verifying the well-formedness of a term as type-checking.

Well-formedness of a term is defined by the rules in Figure 1. Rule **WF-VAR** states that a variable x is well-formed if it is well-scoped. Rule **WF-TOP** states that the term `Top` is always well-formed. Rule **WF-FUN** states that a function $\lambda x \leq t.u$ is well-formed if and only if both its body u and its type annotation t are well-formed. Rule **WF-APP** states that an application uv is well-formed if u is a subtype of a function $\lambda x \leq t.\text{Top}$, such that v is a subtype of t . There are two challenging things to do when type-checking an application: (i) finding a suitable term t , and (ii) proving the subtype relations prescribed by the rules. As for (i), finding such t is challenging since the rule **WF-APP** is not syntax-directed. The algorithm described in [12] finds such t under certain conditions, but in general, this algorithm does not terminate. As for (ii), proving the subtype relation is undecidable in general, since it is undecidable to check if two terms are equivalent, which is a subproblem of checking if they are subtypes.

Term well-formedness

$$\begin{array}{c} \boxed{\Gamma \vdash t \text{ wf}} \\ \frac{x \leq t \in \Gamma}{\Gamma \vdash x \text{ wf}} \text{ WF-VAR} \quad \frac{}{\Gamma \vdash \text{Top wf}} \text{ WF-TOP} \\ \frac{\Gamma \vdash t \text{ wf} \quad \Gamma, x \leq t \vdash u \text{ wf}}{\Gamma \vdash \lambda x \leq t.u \text{ wf}} \text{ WF-FUN} \\ \frac{\Gamma \vdash u \text{ wf} \quad \Gamma \vdash v \text{ wf} \quad \Gamma \vdash t \text{ wf}}{\Gamma \vdash u \leq \lambda x \leq t.\text{Top} \quad \Gamma \vdash v \leq t} \text{ WF-APP} \\ \Gamma \vdash uv \text{ wf} \end{array}$$

Figure 1: Well-formedness rules in PSS

Now we describe subtyping in PSS, which is collected by the rules in Figure 2. These rules describe two reduction relations: a *subtype reduction*, which models promotion steps, and an *equivalence reduction*, which models β -reduction.

Rule **LEQ-DEF** defines the subtyping relation ($\leq$) such that a is a subtype of b if and only if there exists a term c such that a is a subtype of c and b is equivalent to c . We represent in the rest of the paper the subtype reduction vertically and the equivalence reduction horizontally, such that the diagram below represents the subtyping relation $a \leq b$:

$$\begin{array}{ccc} b & \xrightarrow{\equiv} & c \\ & \searrow \leq & \uparrow \\ & & a \end{array}$$

We now describe the subtype reduction and the equivalence reduction. Rule **EQ-REFL** ensures that the equivalence reduction relation is reflexive. Rule **EQ-BETA** instruments the β -reduction that can happen in any context $C \equiv$ thanks to rule **EQ-CTX**. Rule **SUB-TOP** states that any term is a subtype of `Top`. Rule **SUB-PROM** states that if $x \leq t \in \Gamma$ (meaning x has subtype t in context Γ), then x is a subtype of t . Rule **SUB-EQ** ensures that any equivalence reduction is also a subtype reduction. Rule **SUB-CTxFUN** states that a subtype reduction can happen in the body of a function, with its variable x

⁴The fact that well-formedness implies type-safety is an open problem since the introduction of PSS by Hutchins [12], which we are actively investigating in a separate work.

linked to the subtype annotation t in the context Γ . Rules SUB-Top, SUB-Prom, SUB-EQ and SUB-CTXFUN also happen in a context $C_{\leq}$ thanks to rule SUB-CTX.

$$\begin{array}{c}
\frac{\Gamma \vdash a \xrightarrow{\leq} c \quad \Gamma \vdash b \xrightarrow{\equiv} c}{\Gamma \vdash a \leq b} \text{LEQ-DEF} \quad \frac{}{\Gamma \vdash u \xrightarrow{\equiv} u} \text{EQ-REFL} \\
\\
\frac{}{\Gamma \vdash (\lambda x \leq t.u) v \xrightarrow{\equiv} u[x \setminus v]} \text{EQ-BETA} \\
\\
\frac{\Gamma \vdash u \xrightarrow{\equiv} v}{\Gamma \vdash C_{\equiv}[u] \xrightarrow{\equiv} C_{\equiv}[v]} \text{EQ-CTX} \quad \frac{}{\Gamma \vdash u \xrightarrow{\leq} \text{Top}} \text{SUB-Top} \\
\\
\frac{x \leq t \in \Gamma}{\Gamma \vdash x \xrightarrow{\leq} t} \text{SUB-Prom} \quad \frac{\Gamma \vdash u \xrightarrow{\equiv} v}{\Gamma \vdash u \xrightarrow{\leq} v} \text{SUB-EQ} \\
\\
\frac{\Gamma, x \leq t \vdash u \xrightarrow{\leq} v}{\Gamma \vdash \lambda x \leq t.u \xrightarrow{\leq} \lambda x \leq t.v} \text{SUB-CTXFUN} \\
\\
\frac{\Gamma \vdash u \xrightarrow{\leq} v}{\Gamma \vdash C_{\leq}[u] \xrightarrow{\leq} C_{\leq}[v]} \text{SUB-CTX} \\
\\
C_{\equiv} ::= \square \mid (\lambda x \leq C_{\equiv}.t) \mid (C_{\equiv} \leq t) \mid (C_{\equiv} t) \mid (t C_{\equiv}) \mid \\
(Y C_{\equiv}) \mid \text{Or}(C_{\equiv}, t) \mid \text{Or}(t, C_{\equiv}) \\
C_{\leq} ::= \square \mid (C_{\leq} t) \mid (t C_{\leq}) \mid (Y C_{\leq}) \mid \text{Or}(C_{\leq}, t) \mid \text{Or}(t, C_{\leq})
\end{array}$$

Figure 2: Subtyping rules in PSS

Our system is slightly different from the original PSS theory in the following ways: In the original PSS theory, the type system allows terms to be promoted only in the following context: $C_{\text{PSS}} ::= \square \mid (\lambda x \leq t.C_{\text{PSS}}) \mid (C_{\text{PSS}} t)$. In this paper, we extend the original PSS contexts with $(t C_{\leq})$, on which we will elaborate in a moment.

We disallow the following context, $(\lambda x \leq C_{\leq}.t)$. The original theory is invariant; therefore, we keep the invariant property in our theory.

The new allowed context is $(t C_{\leq})$. We allow promotions to happen in this new allowed context, under certain conditions, to allow more expressivity for the programmer. The condition is the following: for this new allowed context to be used in a promotion step, there must be no instances of variables in any subtype annotation anywhere in the term t . The above condition is an over-approximation of the actual set of terms for which it is problematic to promote in this new allowed context. The above condition is also a major change to the theory, as simple terms such as $\lambda x \leq \text{Top}. \lambda y \leq x.y$ are disallowed in some promotion steps. Under the constraint described above, no forbidden promotions can happen; as a result, we conjecture that the metatheory is still correct. All the examples of Section 4 do follow the constraint mentioned

above, where this new allowed context is used extensively, which is crucial for the expressivity of the theory.

Moreover, because the programmer drives the subtyping process, allowing more contexts and therefore making these derivation rules less algorithmic is not problematic; yet it is useful for the programmer to have increased expressivity.

2.2 How we represent objects in this theory

To program in this theory, we need to represent some objects. We either add primitive operators to the theory or encode the objects within the calculus when possible. This section presents all the different objects.

General recursion can be implemented with the aid of a fixed point combinator. Such a fixed point combinator could be written in our calculus thanks to a family of looping terms [12, p. 46]. Such a family results in a term 40 pages long [10]. Therefore, in order to simplify the implementation, we add a primitive fixed point combinator to the theory whose typing and reduction rules are the following:

$$\frac{}{\Gamma \vdash Y f \xrightarrow{\equiv} f(Y f)} \text{EQUIV-Y} \\
\\
\frac{\Gamma \vdash f \text{ wf} \quad \Gamma \vdash t \text{ wf} \quad \Gamma \vdash f \leq \lambda x \leq t. \text{Top} \quad \Gamma \vdash Y f \leq t}{\Gamma \vdash Y f \text{ wf}} \text{WF-Y}$$

We add a primitive variant type to the theory, which represents a possible choice between two or more terms. Given two terms a and b , the variant of these two terms is written $\text{Or}(a, b)$ and indicates a choice between a and b . We add the above construction to model a domain of interpretation of possible values, in the spirit of abstract interpretation. An area of future work would be to add more similar constructs to enrich the expressivity of the language, for instance a set difference operator. Figure 3 presents our encoding of integers with our variant operator being crucial to the encoding.

The subtyping and well-formedness rules of variants are the following:

$$\begin{array}{c}
\frac{\Gamma \vdash a \leq b}{\Gamma \vdash a \leq \text{Or}(b, c)} \text{LEQ-ORR1} \quad \frac{\Gamma \vdash a \leq c}{\Gamma \vdash a \leq \text{Or}(b, c)} \text{LEQ-ORR2} \\
\\
\frac{\Gamma \vdash a \leq c \quad \Gamma \vdash b \leq c}{\Gamma \vdash \text{Or}(a, b) \leq c} \text{LEQ-ORLEFT} \\
\\
\frac{\Gamma \vdash f a \xrightarrow{\equiv} a' \quad \Gamma \vdash f b \xrightarrow{\equiv} b'}{\Gamma \vdash f \text{Or}(a, b) \xrightarrow{\equiv} \text{Or}(a', b')} \text{EQ-OR} \\
\\
\frac{\Gamma \vdash a \text{ wf} \quad \Gamma \vdash b \text{ wf}}{\Gamma \vdash \text{Or}(a, b) \text{ wf}} \text{WF-OR}
\end{array}$$

To preserve the expressivity of variants, in the implementation, we disallow any equivalence reduction that would duplicate a variant. For instance, consider the term $(\lambda x \leq \text{Top}. x x) (\text{Or}(a, b))$, which represents the choice between $(\lambda x \leq \text{Top}. x x) a$ (which is $(a a)$) and $(\lambda x \leq \text{Top}. x x) b$ (which is $(b b)$). If we β -reduce the term $(\lambda x \leq \text{Top}. x x) (\text{Or}(a, b))$ to $(\text{Or}(a, b) \text{Or}(a, b))$, the resulting

expression can lead to terms like $(a\ b)$, which were not possible outcomes of the original expression that only allowed $(a\ a)$ or $(b\ b)$. In fact, the term obtained after the β -reduction $(\text{Or}(a, b)\ \text{Or}(a, b))$ is a supertype of the original term $(\lambda x \leq \text{Top}.x\ x)\ (\text{Or}(a, b))$. Therefore, because the duplication of a variant is similar to a promotion in our calculus, we disallow any duplication of a variant during equivalence steps: if an equivalence reduction step duplicates a variant object, then it is a subtype reduction step, and not an equivalence reduction step.

For all other objects, we use a slight variation of traditional Scott encoding for the terms [8, 18], and we use an encoding reminiscent of algebraic data types for the types [4]. We first present these encodings, and then discuss our choices.

For terms, other encodings exist, but we use the Scott encoding to let the programmer pattern match on the objects they manipulate, which makes practical programming with these objects easier. For instance, the integers 0 and 1 are represented in Figure 3.

$$\begin{aligned} \text{int} &= \lambda f \leq \text{Top}.\lambda z \leq \text{Top}.\lambda s \leq (\lambda m \leq f.\text{Top}).\text{Or}(z, s\ f) \\ \text{zero} &= \lambda z \leq \text{Top}.\lambda s \leq (\lambda m \leq \text{int}.\text{Top}).z \\ \text{one} &= \lambda z \leq \text{Top}.\lambda s \leq (\lambda m \leq \text{int}.\text{Top}).s\ \text{zero} \end{aligned}$$

Figure 3: Scott encoding of integers

With the above encoding, the function *match*, which takes an integer and two choices, one for the case where n is "zero" and one for the other case, is defined as follows:

$$\text{match} = \lambda n \leq \text{int}.\lambda z \leq \text{Top}.\lambda f \leq (\lambda m \leq \text{int}.\text{Top}).n\ z\ f$$

The successor function is defined as follows:

$$\text{succ} = \lambda n \leq \text{int}.\lambda z \leq \text{Top}.\lambda s \leq (\lambda m \leq \text{int}.\text{Top}).s\ n$$

To write the "type" of our objects, we use an algebraic data type encoding to exactly model the possible values of the object: only terms that follow this algebraic data type encoding will be subtypes of this type. For instance, integers are defined recursively as either 0 or a successor of themselves. As a result, our type *int* models all these possible values thanks to our variant construct described in Section 2.2. Our approach is similar to what an iso-recursive type system [16] would be, in the sense that in practical use, the programmer needs to indicate to the type-checker where to fold and unfold the recursive type definition. Figure 4, for instance, shows that *one* is a subtype of *int* thanks to one unfolding, which in our system is an expansion of the fixed point combinator. Our encoding of recursive types resembles the conventional algebraic data type definitions, where $\mu\alpha.\lambda z.\lambda s.z + s\ \alpha$ turns into $\lambda f \leq \text{Top}.\lambda z \leq \text{Top}.\lambda s \leq (\lambda m \leq f.\text{Top}).\text{Or}(z, s\ f)$.

We now explain our differences from the traditional Scott encoding.

The first difference is that the traditional encoding takes a carrier, and ours does not. For instance, the term *zero* is often represented as follows:

$$\lambda c \leq \text{Top}.\lambda z \leq c.\lambda s \leq (\lambda m \leq \text{int}.c).z$$

While we acknowledge that letting the programmer provide a carrier is useful for automatic type-checking, we are not interested in automatic type-checking, and as such, our terms do not take a

carrier as a parameter as it lengthens every term, which makes the process of driving the type-checker harder. Removing the carrier has other advantages, especially with booleans, one of which is described at the end of this section.

The second difference from the traditional Scott encoding is our type *int*. In typed lambda calculus, when the Scott encoding is used, the integer type (with a carrier) is usually represented as follows:

$$\text{wrongint} = \lambda f \leq \text{Top}.\lambda c \leq \text{Top}.\lambda z \leq c.\lambda s \leq (\lambda m \leq f.c).c$$

The above Scott integer type *wrongint* is pathological for our use case for three reasons: First, some terms, which are not integers, are subtypes of this term. For instance, consider the following term:

$$\text{notanint} = \lambda c \leq \text{Top}.\lambda z \leq c.\lambda s \leq (\lambda m \leq \text{wrongint}.c).s\ \text{wrongint}$$

By promoting the variable s to its subtype annotation, the body of the function becomes $(\lambda m \leq \text{wrongint}.c)\ \text{wrongint}$ which β -reduces to *wrongint*, and as such we have that *notanint* is a subtype of *wrongint*.

Second, addition is not commutative on these "incorrect" integers. For instance, consider the standard definition of addition for Scott integers, which takes two parameters, pattern matches on the first one to either return the second parameter if the first parameter is 0, or return the successor of self applied to some arguments. Because of the Scott encoding, the way to pattern match on the first argument is to apply it to both what should be done for the 0 case, and for the successor case. Consider therefore the term *plus wrongint one*, this term is actually *wrongint* applied to some other terms specifying what to do if it is 0 or a successor, but in any case the term *wrongint* always returns the carrier c when applied. The term *plus wrongint one* hence reduces to *wrongint*. Whereas, in contrast, the term *plus one wrongint* pattern matches on *one* to then return the successor of *wrongint*, which is *notanint*. This example of non-commutativity can cause confusion for the programmer because these terms do not have the expected behaviour that we expect from integers.

Third, with the above integer type *wrongint*, unexpected behaviours arise when *wrongint* is used as a term. Consider for instance the following function:

$$\lambda n \leq \text{wrongint}.\text{match}\ n\ \text{one}\ (\lambda m \leq \text{wrongint}.\text{one})$$

This function matches on the value of n , and then returns *one* both if n is "zero" or if n is a successor. However, applied to *wrongint*, for the same reason as the non-commutativity of the addition, it returns its carrier instead of *one*, which is not the expected behaviour. However, if we consider the following term, which is the same term but requires *int* as a parameter instead of *wrongint*:

$$\lambda n \leq \text{int}.\text{match}\ n\ \text{one}\ (\lambda m \leq \text{int}.\text{one})$$

Here, the use of *int* instead of *wrongint*, does not have this issue and with which we can prove with our type-checker that the result is always *one*.

For all these reasons, the encoding we use is the one depicted in Figure 3.

Boolean terms are represented similarly as follows:

$$\begin{aligned} \text{true} &= \lambda t \leq \text{Top}.\lambda f \leq \text{Top}.t \\ \text{false} &= \lambda t \leq \text{Top}.\lambda f \leq \text{Top}.f \end{aligned}$$

$$\begin{array}{c}
\text{int} \xrightarrow{\equiv} \text{Or}(\text{zero}, \text{succ Or}(\text{zero}, \text{succ int})) \xrightarrow{\equiv} \text{Or}(\text{zero}, \text{Or}(\text{succ zero}, \text{succ succ int})) \xrightarrow{\equiv} \text{Or}(\text{zero}, \text{Or}(\text{one}, \text{succ succ int})) \\
\uparrow \leq \\
\text{one}
\end{array}$$

Figure 4: *one* is a subtype of *int*

The type for booleans is initially defined in a style similar to integers:

$$\text{notyetbool} = \lambda f. \lambda t. \text{Or}(t, f)$$

Because the term *notyetbool* is lengthy, in the implementation, we use the following type, which is equivalent, simpler, and more practical:

$$\text{bool} = \text{Or}(\text{true}, \text{false})$$

The lack of a carrier in booleans allows more expressivity; one can therefore build an if-expression where each branch returns a different type (for instance, one branch returns *int* whereas the other returns *succ*), and as long as the program ultimately type-checks, the program is considered valid.

It is left to future work to define other usual objects such as lists, pairs, etc. We now present the differences between the original PSS theory introduced in [12] and the theory we use.

Our theory enables a limited form of abstract interpretation. For instance, if a function *f* is a subtype of $\lambda n \leq \text{int}. \text{succ int}$, then for any input *x* with $x \leq \text{int}$, the result *f x* is a subtype of $(\lambda n \leq \text{int}. \text{succ int}) x$ which reduces to *succ int*. In essence, the type of a term is a superset of the set of possible values of the term.

The subtyping relation in our system provides some information about the term structure. For instance, if we establish that $u \leq \text{succ int}$, then we can deduce that *u* is in essence a successor when it comes to the usual pattern matching of integers. Consider for instance a function $f = \lambda n \leq \text{int}. \text{match } n \text{ zero } (\lambda n \leq \text{int}. \text{one})$ which takes an integer, matches on its value, returns *zero* if the integer is 0, and else returns *one*. Then we know that *f u* is a subtype of *f (succ int)*, which returns *one*. This reasoning means that there is no possible value for *u* such that the call of *f* on *u* leads to the case "zero", which means that in practice *u* is a successor. Therefore once the factorial function has been proven be a subtype of *succ int*, it is safe to divide by its result.

It is unclear how general this method is to represent set of objects. For instance, we believe it is impossible to represent the set of prime number in such fashion. We can represent the set of non-prime, but as the calculus doesn't have a "difference" operator this doesn't lead to a construction of the set of prime numbers.

3 The interactive type-checker

In this section, we discuss the technical choices related to the implementation, and we discuss how to use the type-checker.

3.1 Why Emacs

In order to write our type-checker, we had to choose a language and a framework. To prove a subtype relation, the programmer selects a subterm of a program and chooses some action on it. The way for the programmer to do so is to select a subterm on which they want to take action, and then select the desired action. This

requires exposing to the programmer the exact AST of the program. Emacs and Lisp are technological solutions that adapt very well to this interaction scheme. We now describe the syntax of our calculus in Lisp.

3.2 Syntax of our calculus in Lisp

We represent PSS terms by the following grammar:

1	<code>'x</code>	; A variable "x"
2	<code>top</code>	; The term Top
3	<code>(FUN x u v)</code>	; A function with type u and body v
4	<code>(u v)</code>	; An application: u applied to v
5	<code>(Y u)</code>	; Fixed point combinator applied to u
6	<code>(Or u v)</code>	; The term Or(u, v)

We use the traditional Lisp conventions. Terms are symbols, written within asterisks, for instance the term *zero* is written `*0*`, and the addition is written `*plus*`. Variables in terms are always quoted, otherwise they are considered as symbols that may be defined elsewhere, for instance the term `(FUN n *int* 'n)` is valid, but the term `(FUN n *int* n)` is not.

We now write with these conventions some of the Scott integers we described in Section 2.2 in Figure 3:

```

1 (define-term '*0* ; The integer 0
2   '(FUN zero-case top
3     (FUN succ-case (FUN n *int* top)
4       'zero-case)))
5
6 (define-term '*1* ; The integer 1
7   '(FUN zero-case top
8     (FUN succ-case (FUN n *int* top)
9       ('succ-case *0*))))
10
11 (define-term '*int* ; The type int
12   '(y (FUN self top
13     (FUN zero-case top
14       (FUN succ-case (FUN n 'self top)
15         (OR 'zero-case ('succ-case 'self)))))))
16
17 (define-term '*succ* ; The successor function
18   '(FUN n *int*
19     (FUN zero-case top
20       (FUN succ-case (FUN n top top)
21         ('succ-case 'n))))

```

3.3 Type-checking programs

To type-check a given program, there are two phases:

- (1) Check that the program is well-formed with the rules described in Figure 1 in an automatic process that generates a list of *proof obligations* to resolve.

- (2) Address these proof obligations with the help of commands described in Figure 5.

To check that a program is well-formed, we use the rules described in Figure 1. Our algorithm traverses the program to look for all applications of the form $u v$. When we encounter such an application, the type-checking process generates two proof obligations with a *placeholder* that stands for the t in the premises of the rule WF_APP . The two proof obligations are to prove that $u \leq \lambda x \leq t$. Top and that $v \leq t$.⁵ These proof obligations to resolve are displayed in the buffer ***Obligations***, with the following format, where a is the term to prove as a subtype of the objective b :

```
(=<p a b) ; A proof obligation that a <= b
```

In Emacs, when the programmer addresses a proof obligation, we spawn the following buffers:

- ***Term*** which displays the current status of the term a .
- ***Objective*** which displays the current status of the objective b .
- ***Goal*** which displays the list of current *goals*, which are proof obligations, to prove.
- ***Temp-Proof*** which holds the current interactive proof done so far, the programmer's actions are logged to ***Temp-Proof*** until a commit is made to allow exploration and rollbacks in the proof process.
- ***Proof*** which holds the consolidated proof, which thanks to the commit command, C-c c, pastes all the content of the buffer ***Temp-Proof*** to the buffer ***Proof***. At any time the programmer can recompute the current state of the buffers ***Term***, ***Objective*** and ***Goal*** according to the consolidated proof displayed in the buffer ***Proof***, with the command C-c r.

Then, the programmer needs to pick a subterm of either a or b on which they want to perform an action, and which action they wish to execute. Only equivalence reductions are allowed on the term b , whereas on the term a the programmer can do both equivalence and subtype reductions. The proof ends when both terms are equal. The proof obtained is the path in the form of a diagram as the one in Section 2.1 from the term a to the term b .

3.4 Identification of subterms

An action made by the programmer acts on a subterm of a term. A *path* is a list of integers that describes the position of the subterm on which the action is done, from the main term, which can be used to traverse the AST to find the wanted subterm. At each node of the tree, the child to take is indicated by the integer at the current position in the path, where the first child is at position 0, the second child is at position 1, etc. For instance, take the term $((a b) c)$, then the path **nil** indicates the whole term, the path (0) indicates the term $(a b)$, and the path $(0\ 1)$ indicates the term b .

3.5 Normalisation of terms

The command C-c B normalises a term with only β -substitution steps. Indeed, it is important that the C-c B command does not

⁵We know that if u is already a function, then the term t to use is its type annotation thanks to the inversion result in [12]. This improvement is implemented.

• Proof mode commands:

- C-c S: Compile a term
- C-c s: Start to address a proof obligation
- C-c m: Merge ***Temp-Proof*** in ***Proof***
- C-c r: Recompute ***Term***, ***Objective*** and ***Goal*** according to ***Proof***
- C-c q: Qed command, indicate the proof as ended

• Action commands (Manipulation of term that modifies the proof, available on both ***Term*** and ***Objective*** unless specified otherwise):

- C-c b: Equivalence reduction at point
- C-c B: Beta substitution (recursive) at point
- C-c e: Macro expand at point
- C-c p: Promote at point only in ***Objective***
- C-c o: Split "or" at point only in ***Term***
- C-c C-o n: Replace the "or" variant by its n choice at point only in ***Objective***

Figure 5: List of Emacs commands of our type-checker

make unwanted actions such as duplicating a variant as explained in Section 2.2, or reducing a fixed point combinator which would likely lead to an infinite reduction sequence. For these reasons, the command C-c B is limited to only β -reductions.

When the programmer wishes to reduce a fixed point combinator, or to duplicate a variant in a controlled manner, they can use the C-c b command, which is a single step of equivalence reduction with no limitation.

3.6 How to deal with variants

Two different actions can be made for variants, depending on whether the variant occurs in the term, or in the objective.

If a programmer wishes to prove that a term $Or(a, b)$ is a subtype of a term c , the programmer can split the left variant with the command C-c o, which spawns two subgoals, one for each element of the variant, that the programmer must prove separately.

If a programmer wants to prove that a term a is a subtype of a term $Or(b, c)$, then they can choose whether they wish to prove that the term a is a subtype of b or of c , with either the command C-c C-o 0 to choose b , or C-c C-o 1 to choose c .

3.7 Recursive programs

In our system, recursive terms are built with our fixed point combinator, that has a well-formedness rule stated in Section 2.2. According to this rule, in order for a recursive term to be well-formed, one should provide a term t as the rule is not syntax-directed. The term t is provided by the programmer when they write a recursive term. For instance, the factorial function is:

```
1 (define-term '*factorial*
2   '(y (FUN self (FUN n *int* (*succ* *int*))
3     ...
```

In the above function, the term t required by the well-formedness rule is $\lambda n \leq int. succ\ int$.

In contrast to other dependent type theories, recursive terms do not have to be terminating. The Syracuse function defined in

Section 4.5, for instance, potentially is not terminating, yet is well-formed.

There can be alternative definitions of recursive terms, for instance the factorial function can be defined with the term $t' = \lambda n \leq \text{int}. \text{int}$ and also be well-formed. However, the term t of the rule WF-Y is the set of values that the function returns: indeed, one of the premises of the rule WF-Y is $\Gamma \vdash Yf \leq t$ which when t is badly chosen (for instance $t = \text{Top}$), provides no useful information about the result of the function. For the factorial case, the motivating example of the paper is proving that its result is greater than one, hence the choice of t being $\lambda n \leq \text{int}. \text{succ int}$ which ensures the result is greater than one.

4 Example of addressing proof obligations

This section presents different examples to showcase how the programmer addresses different proof obligations.

We begin by proving that $*1*$ is a subtype of $*\text{int}*$. This example serves both as an example of our recursive types, as well as showing how to deal with simple type-checking of integers.

Then we prove that the addition of an integer greater than 1, with any integer, results in an integer greater than 1. This example showcases that reasoning about integers and their sets of values is possible with our system.

Next, we consider the factorial function, and prove it always returns a result greater than 1. This example showcases both reasoning about a recursive function, and some arithmetic reasoning.

Finally, we arrive at our initial example of safe division. This example showcases the generation of proof obligations when a program is type-checked.

Additionally, we prove the Syracuse function always returns 1. This example showcases that our approach works for non-primitive recursive functions.

4.1 Proving that $*1*$ is a subtype of $*\text{int}*$

The term $*1*$ is defined as follows:

```
1 (define-term '*1*'
2   '(FUN zero-case top
3     (FUN succ-case (FUN n *int* top)
4       ('succ-case *0*))))
```

And the term $*\text{int}*$ is defined as follows:

```
1 (define-term '*int*'
2   '(y (FUN self top
3       (FUN zero-case top
4         (FUN succ-case (FUN n *int* top)
5           (OR 'zero-case ('succ-case 'self)))))))
```

In order to unify both terms, we first expand the term $*0*$ in the term $*1*$. We obtain the following term:

```
1 '(FUN zero-case top
2   (FUN succ-case (FUN n *int* top)
3     ('succ-case
4       (FUN zero-case top
5         (FUN succ-case (FUN n *int* top)
6           'zero-case))))))
```

Now we reduce the term $*\text{int}*$ to obtain:

```
1 (FUN zero-case top
2   (FUN succ-case (FUN n *int* top)
3     (OR 'zero-case
4       ('succ-case
5         (y (FUN self top (FUN zero-case top
6           (FUN succ-case (FUN n *int* top)
7             (OR 'zero-case ('succ-case 'self))))))
8       ))))
```

We can now pick the second choice in the first Or disjunction, which selects the successor choice, and then pick the first choice in the second Or disjunction, which selects the "zero" choice. All in all, the result should be the successor of "zero", which is the term $*1*$. Once we select these choices and β -reduce, we obtain:

```
1 (FUN zero-case top
2   (FUN succ-case (FUN n top top)
3     ('succ-case
4       (FUN zero-case top
5         (FUN succ-case (FUN n top top)
6           'zero-case))))))
```

The term above is exactly the reduced term $*1*$, which concludes the proof.

The proof trace, where β -reduction and macro expansion are removed for readability, is the following:

```
1 (start-term *1*)
2 (start-goal *int*)
3 ;; reduce fixed point combinator in *int*
4 (rewrite beta-substitute :on-objectives t
5   :path nil)
6 ;; choice: the successor
7 (rewrite (objective-or-split 1) :on-objectives t
8   :path (3 3))
9 ;; choice: zero
10 (rewrite (objective-or-split 0) :on-objectives t
11   :path (3 3 1 1 3 3 3))
12 ;; reflexivity
13 (qed)
```

4.2 The addition of a successor and anything, is a successor

In this section, we address the following proof obligation: ($\leq_p$ (($*\text{plus}*$ ($*\text{succ}*$ $*\text{int}*$)) $*\text{int}*$)) ($*\text{succ}*$ $*\text{int}*$)).

Our starting term is:

```
1 ((*plus* (*succ* *int*)) *int*)
```

We first discharge the case where n is "zero". To do so, we expand the term $*\text{succ}*$ and execute a step of the function $*\text{plus}*$. We now obtain:

```
1 (*succ*
2   (((y (FUN self (FUN a *int* (FUN b *int* *int*))
3     (FUN m *int*
4       (FUN n *int*
5         (('m 'n) (FUN a *int* (*succ* (('self 'a)
6           'n)))))))
7     *int*)
8   *int*))
```

The above term is the expected result; the function `*plus*` was applied to a successor, so according to its definition the result is the successor applied to itself with some arguments. In the term above, the recursive call is `((self 'a) n)`, and we wish here to only prove that the result is a successor. We can therefore promote the induction call to only an integer, and still obtain the correct result.

To do so, we promote the variable `'self`, and we can then apply the β -reduction to obtain:

```
1 (*succ*
2   ((int*
3     *int*)
4     (FUN a int* (*succ* int*))))
```

The above term is in fact a pattern matching on the value of `*int*`. If the value of `*int*` is `*0*`, then the result is `*int*`, else the result is also `(*succ* int*)`. Once we expand the term `*int*`, we see more clearly the two options, as we obtain after some β -reduction:

```
1 (*succ*
2   (OR int* (*succ* int*)))
```

We now split the two choices, thanks to Rule LEQ-ORLEFT, and conclude each case easily.

4.3 Factorial always returns a result greater than one

In this section, we prove that the factorial function always returns a result greater than 1.

The factorial function is defined as follows:

```
1 (define-term '*factorial*
2   (y (FUN self (FUN n int* (*succ* int*))
3     (FUN n int*
4       (('n
5         ;; n == 0 -> 1
6         *1*)
7         ;; n == S(a) -> n * (fact a)
8         (FUN a int*
9           ((*mult* 'n) (self 'a))))))))))
```

In this section, we prove that `*factorial*` is a subtype of `(*succ* int*)`. To prove the required subtyping derivation, we split the value of `*int*` into either `*0*`, or a successor, thanks to Rule LEQ-ORLEFT. For the case `*0*`, once we expand and β -reduce, we obtain directly the term `*1*`, which we can prove as in Section 4.1 is a subtype of `(*succ* int*)`. Otherwise, we obtain the following term:

```
1 ((y (FUN self (FUN n int* (*succ* int*))
2   (FUN n int*
3     (('n
4       *1*)
5       (FUN a int* ((*mult* 'n) (self 'a)))))))
6   (...))
```

We have omitted the term with `...` because the term is long; the term is beta-equivalent to `(*succ* int*)`. The term obtained above is the application of the factorial function to the term `(*succ* int*)`. To conclude, we first promote the recursive call of the factorial, in order to mimic the induction principle needed to prove

the wanted result. We therefore promote the variable `'self` to its type annotation `FUN n int* (*succ* int*)`, after which the term `(self 'a)` reduces to `(*succ* int*)`. After these changes, and some β -reduction, we obtain the following term:

```
1 ((*mult*
2   (...))
3   (*succ* int*))
```

Here, the term `...` designates the same term as above. The proof trace, up to this point is the following:

```
1 (start-term
2   (*factorial* int*))
3 (start-goal
4   (*succ* int*))
5
6 ;; split int*
7 (rewrite split-or-all :on-objectives nil
8   :path (1 3 3))
9 ;; case int* = 0
10 (focus 0)
11 ;; choice "zero" in the objective
12 (rewrite (objective-or-split 0) :on-objectives t
13   :path (1 1 3 3 3))
14 ;; reflexivity
15 (qed)
16
17 ;; case int* = succ(...)
18 (focus 0)
19 ;; "by induction" - promote self to its objective
20 (rewrite promote :on-objectives nil
21   :path (0 1 3 3 1 3 1 0))
```

To then conclude this proof, one should resolve the obligation (`<=p ((*mult* (*succ* int*)) (*succ* int*)) (*succ* int*)`) as in Section 4.2.

4.4 Type-checking the safe division

In this section, we type-check the safe division program described at the start of the paper. The program we wish to type-check is the following:

```
1 '(FUN p int*
2   (FUN n int*
3     ((*safe-div* 'p) (*factorial* 'n))))
```

When we type-check this program, a list of obligations is generated. Most of them are immediate to prove, except for (`<=p *factorial* (FUN n int* (*succ* int*))`), which we have already addressed in Section 4.3.

Once the above non-trivial proof obligation has been addressed, and once all the other proof obligations are addressed, the program is type-checked and can be executed without any error.

In the next section, we highlight that our approach works with any recursive function, including potentially non-terminating ones, by proving that the Syracuse function returns 1.

4.5 The Syracuse function always returns 1

One important example that highlights the difference between our type-checker and traditional proof assistants is the Syracuse

function defined in the introduction. It is an open problem to know whether the Syracuse function terminates, yet it is straightforward to see that if the Syracuse function terminates then the result must be 1. We define the Syracuse function as follows:

```

1 (define-term '*syracuse*
2   (y (FUN self (FUN n *int* *1*)
3     (FUN n *int*
4       (((*isBelowOne* 'n)
5         *1*)
6         ((*isEven* 'n)
7           ('self ((*div* 'n) *2*)))
8           ('self (*succ* ((*mult* *3*) 'n))))))))))

```

In our implementation, the functions `*isBelowOne*`, `*isEven*` and `*div*` are not yet implemented, which does not impact this example. To type-check the Syracuse function, we need to address the following proof obligation (`<=p *syracuse* (FUN n *int* *1*)`)).

To do so, we promote each recursive instance of `'self` to the objective, both conditions `(*isBelowOne* 'n)` and `(*isEven* 'n)` to a boolean (which in this case depends on their implementation), to obtain the following term:

```

1 ((((*bool* *1*) *bool*) *1*) *1*)

```

We can now split both booleans, thanks to Rule LEQ-ORLEFT, and conclude each case. The proof trace, up until proving that each term `*1*` is indeed `*1*` in each boolean case is the following:

```

1 (start-term
2   (*syracuse* *int*))
3 (start-goal
4   *1*)
5
6 ;; Promote the first recursive call
7 (rewrite promote :on-objectives nil
8   :path (0 1 3 3 1 1 0))
9 ;; Promote the second recursive call
10 (rewrite promote :on-objectives nil
11   :path (0 1 3 3 1 0 1 0))
12 ;; Split the first boolean
13 (rewrite split-or-all :on-objectives nil
14   :path (0 0))
15 ;; 1st boolean case *true*
16 (focus 0)
17
18 ;; Split the second boolean
19 (rewrite split-or-all :on-objectives nil
20   :path (1 0 0))
21
22 ;; Then, the rest of the proof

```

This example highlights the difference between our system and traditional dependent-type systems that have strong normalisation requirements.

5 Conclusion

In this section, we present the related work, the future work, and a conclusion.

5.1 Related work

Pure type systems [15] allow a finer control over the kinds used for terms and types, which generalise the lambda calculi with types in the Barendregt Cube [3]. These systems, however, use a typing relation instead of a subtyping one.

Other attempts try to unify typing and subtyping, like $\lambda I_{\leq}$ [22], a dependently typed generalisation of System $F_{\leq}$. In a different approach, Aspinall studied the combination of subtyping with singleton types [1].

Some systems, like Cedille, use pure lambda calculus to encode types, similar to our approach, instead of using primitive operators for algebraic data types like in other theories [19]. Cedille combines dependent types with a restricted notion of recursion. Our system, however, blurs completely the distinction between terms and types.

Proof assistants like Rocq [7], Agda [5] leverage dependent types to prove properties about programs or mathematical properties in general. In these systems, the programmer specifies a type that captures the desired specification, and then proceeds to construct a term, either via tactics for Rocq, or directly inputting the term for Agda. All of these approaches require strong normalisation, which is not required in our theory.

These proof assistants can be used in Emacs with Proof General [2], which is also written in Emacs Lisp, and lets the programmer interact with a variety of proof assistants in Emacs. The interface exposed to the programmer is similar: multiple buffers are spawned with the relevant information, and are updated in sync when the programmer interacts with them.

Interactive typecheckers for undecidable type theories were also considered in the past, for instance the NuPRL proof development system [13] which is a proof assistant based on the undecidable Martin-Löf intuitionistic type theory [14].

Proof assistants that allow, as our approach, reasoning about non-terminating terms have been studied, with for instance LCF [11, 17], with the drawbacks of the complexity of the reasoning.

Other programming languages with dependent types exist, such as Idris [6] and F* [20, 21], we believe our approach is more versatile as it does not have any strong normalisation requirement on either terms or types.

5.2 Limitations and future work

In order to increase the expressivity of our type-checker, multiple features that are already implemented in the traditional proof assistants could be added to our type-checker. We enumerate some of them.

We are unable to deal with refinement types [9]. One simple example with integers is the set of primes. While we can build the set of non-primes, which is in our calculus $((\text{FUN } x \text{ *int* } ((\text{*mult* 'x' x}) * \text{int*})))$, it is unclear in our system how to build the set of primes.

For the type `int`, the only case disjunction we are allowed to do is to be either 0 or a successor. An area of future work is to allow the programmer to do more advanced case disjunctions, for instance to define a case disjunction on the evenness of an integer.

An area of future work is bringing more automation to our type-checker. Proof assistants usually have a lot of automation features

that are very handy for programmers. Bringing automation to our type-checker is an area of future work.

Even if there is no Curry-Howard correspondence in PSS, some induction principles could be extracted from recursive functions. The recursive function extracted would take an additional *fuel* parameter which decreases at each recursive call, and the extracted induction principle is the strict decrease of this fuel parameter at each recursive call.

5.3 To conclude

In this system description, we described our experimental type-checker based on the PSS theory. We believe the PSS theory and our type-checker have some interesting properties. Our system is more versatile than other type systems, as it allows the programmer to write and reason about non-terminating terms.

References

- [1] David Aspinall. 1994. Subtyping with singleton types. In *International Workshop on Computer Science Logic*. Springer, 1–15.
- [2] David Aspinall. 2000. Proof General: A generic tool for proof development. In *Tools and Algorithms for the Construction and Analysis of Systems: 6th International Conference, TACAS 2000 Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2000 Berlin, Germany, March 25–April 2, 2000 Proceedings 6*. Springer, 38–43.
- [3] Henk Barendregt, S. Abramsky, D. M. Gabbay, T. S. E. Maibaum, and H. P. Barendregt. 1992. Lambda Calculi with Types. (1992), 117–309.
- [4] Richard Bird and Oege De Moor. 1996. The algebra of programming. *NATO ASI DPD* 152 (1996), 167–203.
- [5] Ana Bove, Peter Dybjer, and Ulf Norell. 2009. A brief overview of Agda—a functional language with dependent types. In *Theorem Proving in Higher Order Logics: 22nd International Conference, TPHOLs 2009, Munich, Germany, August 17–20, 2009. Proceedings 22*. Springer, 73–78.
- [6] Edwin Brady. 2013. Idris, a general-purpose dependently typed programming language: Design and implementation. *Journal of functional programming* 23, 5 (2013), 552–593.
- [7] Thierry Coquand and Gérard Huet. 1986. *The calculus of constructions*. Ph.D. Dissertation. INRIA.
- [8] Haskell B Curry, J Roger Hindley, and Jonathan P Seldin. 1972. *Combinatory Logic. Volume II*. North-Holland Publishing Company.
- [9] Tim Freeman and Frank Pfenning. 1991. Refinement types for ML. In *Proceedings of the ACM SIGPLAN 1991 conference on Programming language design and implementation*. 268–277.
- [10] Herman Geuvers and Joep Verkoelen. 2009. On fixed point and looping combinators in type theory.
- [11] Michael J Gordon, Arthur J Milner, and Christopher P Wadsworth. 1979. *Edinburgh LCF: a mechanised logic of computation*. Springer.
- [12] DeLesley Hutchins. 2009. *Pure subtype systems: A type theory for extensible software*. Ph.D. Dissertation. The University of Edinburgh.
- [13] Christoph Kreitz. 2002. The Nuprl Proof Development System, Version 5: Reference Manual and User’s Guide. *Department of Computer Science, Cornell University* (2002), 10.
- [14] Per Martin-Löf. 1984. Intuitionistic type theory vol. 1. (1984).
- [15] James McKinna and Robert Pollack. 1993. Pure type systems formalized. In *Typed Lambda Calculi and Applications: International Conference on Typed Lambda Calculi and Applications TLCA’93 March, 16–18, 1993, Utrecht, The Netherlands Proceedings 1*. Springer, 289–305.
- [16] Benjamin C. Pierce. 2002. *Types and programming languages*. MIT Press.
- [17] Gordon D. Plotkin. 1977. LCF considered as a programming language. *Theoretical computer science* 5, 3 (1977), 223–255.
- [18] Dana Scott. 1962. A system of functional abstraction (1968). Lectures delivered at University of California, Berkeley. *Cal* 63 (1962), 1095.
- [19] Aaron Stump. 2017. The calculus of dependent lambda eliminations. *Journal of Functional Programming* 27 (2017), e14.
- [20] Nikhil Swamy, Juan Chen, Cédric Fournet, Pierre-Yves Strub, Karthikeyan Bhargavan, and Jean Yang. 2011. Secure distributed programming with value-dependent types. *ACM SIGPLAN Notices* 46, 9 (2011), 266–278.
- [21] Nikhil Swamy, Joel Weinberger, Cole Schlesinger, Juan Chen, and Benjamin Livshits. 2013. Verifying higher-order programs with the Dijkstra monad. *ACM SIGPLAN Notices* 48, 6 (2013), 387–398.
- [22] Yanpeng Yang and Bruno C d S Oliveira. 2017. Unifying typing and subtyping. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 1–26.